

Coding Manual for Radial Interaction and Design Pattern Classification

Version: 1.0

Date: 14.02.2026

Authors:

- Aleksandar Anzel, Center for Artificial Intelligence in Public Health Research (ZKI-PH), Robert Koch Institute, Berlin, Germany
- Georges Hattab, Center for Artificial Intelligence in Public Health Research (ZKI-PH), Robert Koch Institute, and Department of Mathematics and Computer Science, Freie Universität Berlin, Germany

Associated manuscript:

“Limitations of Interaction Techniques in Modern Radial Chart Implementations”

Submitted to: *Discover Computing* (Springer Nature)

Purpose of this document:

This coding manual specifies the operational definitions and decision rules used by the authors to categorize:

1. Radial visualization tools/systems by Draper *et al.*'s radial design patterns, and
2. Tools/systems and high-level visualization libraries by Kosara *et al.*'s interaction taxonomy,
for the construction of Table 1 and Table 2 in the associated manuscript.

Taxonomic foundations:

- Draper, G. M., Livnat, Y. & Riesenfeld, R. F. *A survey of radial methods for information visualization*. IEEE Transactions on Vis. Comput. Graph. 15, 759–776, DOI: 10.1109/TVCG.2009.23 (2009).
- Kosara, R., Hauser, H. & Gresh, D. L. *An Interaction View on Information Visualization*. In Eurographics 2003 - STARs, DOI: 10.2312/egst.20031092 (Eurographics Association, 2003).

1. Scope and units

1.1 Taxonomies used

- **Radial design patterns (Draper *et al.*):**
 - Tree
 - Star
 - Concentric
 - Spiral
 - Euler
 - Connected Ring
 - Disconnected Ring
- **Interaction categories (Kosara *et al.*, specialized for radial):**
 - Focus+Context
 - Multiple Views
 - Interaction with Dimensions
 - Domain Specific Methods

1.2 Units of coding

We distinguish two related but slightly different coding tasks:

1. Tools/systems (Table 1)

- Unit: an individual radial visualization tool/system described in a paper or software artifact.
- Variables:
 - Dominant radial pattern (fixed from Draper; not re-coded here).
 - Presence of each interaction category: yes/no (Full / N/A).

2. Libraries (Table 2)

- Unit: a (library, radial pattern, interaction category) triple.
- Variables:
 - Support level: Full , Partial , or N/A .

The same operational definitions of patterns and interaction categories are used in both tasks; only the **support level scale** differs (yes/no vs Full/Partial/N/A).

2. General coding workflow

1. Identify the unit

- Tools: pick one named system (*e.g.*, “Daisy”, “StarCoordinates”).

- Libraries: pick one library (*e.g.*, Plotly, Altair/Vega Lite, ggplot2) and a specific radial pattern it supports (*e.g.*, Star, Concentric).

2. Gather evidence

- For tools: read the primary paper (and any supplemental documentation) and examine all figures and interaction descriptions.
- For libraries: consult the official documentation, tutorials, and gallery examples for radial charts (polar, sunburst, pie, radar, *etc.*).

3. Determine radial pattern (tools only)

- Assign the tool to one Draper pattern using the rules in Section 3 (Tree, Star, Concentric, Spiral, Euler, Connected Ring, Disconnected Ring). Tools already mapped by Draper are taken as given unless strong evidence suggests otherwise.

4. Assess interaction categories

- For each of the four interaction categories, evaluate whether the tool/library implements them and at what level, using the decision rules in Sections 4—5.

5. Record codes

- Tools: for each tool × interaction category, code **Full** (yes) or **N/A** (no).
- Libraries: for each library × pattern × interaction category, code **Full**, **Partial**, or **N/A**.

6. Notes

- Coders should record short notes when decisions are borderline or required inference from ambiguous descriptions (*e.g.*, “Multiple Views only via static small multiples, no linking described”). These notes support later disagreement resolution.

3. Radial design patterns (Draper) — decision rules

These rules are used explicitly when coding tools; for libraries, patterns are inferred from chart types (*e.g.*, pie vs sunburst).

For each tool/library example, coders answer: “Which pattern does this radial visualization most closely follow?”

3.1 Tree pattern (Tree)

- **Definition:** Hierarchical structures with branching elements radiating from the center (root in center; nodes along branches).
- **Visual cues:** clear tree structure, edges from parent to children, radial layout.
- **Examples:** Hyperbolic Browser, MoireGraphs, Visual Thesaurus, RINGS.
- **Rule:** If edges and hierarchy structure dominate and nodes emanate from a central root, code as `Tree`.

3.2 Star pattern (Star)

- **Definition:** Simple spoke like arrangements without branching; multiple axes radiating from a common center (e.g., star plots, radar charts).
- **Visual cues:** each axis is an independent dimension; no hierarchical levels.
- **Examples:** Star Coordinates, radar charts, many polar bar/pie arrangements used as multivariate plots.
- **Rule:** If the structure is “hub and spokes” with no explicit hierarchy, code as `Star`.

3.3 Concentric pattern (Concentric / radial space filling)

- **Definition:** Ring based layouts for hierarchical data, often called sunburst or radial space filling diagrams.
- **Visual cues:** concentric rings represent depth in the hierarchy; inner ring is root level.
- **Examples:** Sunburst, InterRing, StoryPrint, Radial Traffic Analyzer.
- **Rule:** If hierarchy levels are encoded as rings with areas/segments filling the space, code as `Concentric`.

3.4 Spiral pattern (Spiral)

- **Definition:** Continuous spiral arrangements, commonly for time series or periodic data.
- **Visual cues:** one or more spirals; data mapped along spiral arms; often used for temporal patterns.
- **Examples:** SpiraClock, RankSpiral, SpiraGraph.
- **Rule:** If the main structure is a spiral and the data follow the spiral path spatially, code as `Spiral`.

3.5 Euler pattern (Euler)

- **Definition:** Radial layouts representing set relationships and overlaps (Euler/Venn like structures), often with interactive lenses or regions—not simply

polar bar charts.

- **Visual cues:** overlapping regions/areas inside or around a larger circle; emphasis on set membership and overlap.
- **Examples:** Zoomology, MoireTrees, Bubbleworld.
- **Rule:** Only code as `Euler` if the design clearly implements Euler or Venn like region overlaps in radial form; do **not** code standard polar bar charts as Euler.

3.6 Connected Ring pattern (ConnectedRing)

- **Definition:** Nodes arranged on one or more rings with explicit edges (links) connecting them; often for network or relational data.
- **Visual cues:** ring of nodes, arcs/edges across or between rings; emphasis on relationships.
- **Examples:** Daisy, NetMap, VisAlert, various genome circular plots with links.
- **Rule:** If circular rings of nodes are present and inter node links are primary, code as `ConnectedRing`.

3.7 Disconnected Ring pattern (DisconnectedRing)

- **Definition:** Circumferential arrangements without explicit connections; often event sequences or categories arranged around a circle.
- **Visual cues:** rings or annuli with marks/events, but no explicit network edges.
- **Examples:** HoopDiagram, EventTunnel.
- **Rule:** If elements are arranged in rings without explicit linking edges, code as `DisconnectedRing`.

3.8 Hybrid/ambiguous cases

- When a visualization could fit multiple patterns, coders choose the **dominant pattern as described by the authors** (if stated). If not stated, choose the pattern that best matches the primary structural metaphor (e.g., if both rings and explicit edges are central, prefer `ConnectedRing` over `Concentric`).

4. Interaction categories — general decision logic

For each tool or library example, coders answer the following, using evidence from papers or documentation:

1. Does the system/library provide **Focus+Context** interaction for this radial view?
2. Does it provide **Multiple Views** (coordinated views)?

3. Does it provide **Interaction with Dimensions**?

4. Does it provide **Domain Specific Methods**?

- For **tools**, answer Full (yes) or N/A (no).
 - For **libraries**, decide Full / Partial / N/A using the specific rules below.
-

5. Focus+Context

5.1 Definition

Interaction techniques that provide detailed information about a selected focus region while preserving the overall context of the visualization.

Subtypes:

- **Distortion oriented methods:** fisheye, hyperbolic projections, continuous zoom that keeps the whole structure visible.
- **Overview methods:** mini maps, secondary overview panels.
- **Filtering techniques:** interactive filtering that keeps a sense of overall structure.
- **In place methods:** highlighting, tooltips, details on demand overlays within the same view.

5.2 Tools/systems: yes vs no

- **Code Focus+Context = Full (yes) if**
 - The paper describes any interactive focus mechanism where part of the radial display is emphasized (e.g., lens, distortion, detail on demand) while the full structure remains visible.
 - OR hovering, clicking, or selecting an item yields additional detail or highlighting in place, without losing context.
- **Code Focus+Context = N/A (no) if**
 - Only static images are shown, or interaction is limited to non contextual view switching (e.g., opening a completely separate window with details and no overview).

5.3 Libraries: Full vs Partial vs N/A

For a given library and pattern:

- **Full**
 - The library provides documented, built in Focus+Context behavior for radial charts (e.g., drill down sunburst with persistent breadcrumb/overview, interactive tooltips on radial segments, lens/distortion for radial views) via high level options.
 - Users can activate it with minimal configuration (properties, parameters), without writing low level rendering code.
 - **Partial**
 - Some Focus+Context behavior is possible but limited or not fully integrated. Examples:
 - Tooltips on radial charts but no structural focus+context (e.g., no dedicated fisheye or drill down framework).
 - Focus behavior requires combining generic primitives (e.g., overlays) with moderate custom code.
 - **N/A**
 - No documented way to implement Focus+Context specifically for radial charts beyond generic static zoom/pan, or only through low level hacks outside the high level API.
-

6. Multiple Views

6.1 Definition

Two or more coordinated visualizations (views) where interactions in one view (selection, filtering, zoom) affect at least one other view.

6.2 Tools/systems: yes vs no

- **Code Multiple Views = Full (yes) if**
 - The system shows multiple radial or mixed views of the same data and explicitly supports linking (e.g., brushing, synchronized selection, shared filtering).
 - Examples: DataMeadow, CircleView, polar diagrams with linked small multiples.
- **Code Multiple Views = N/A (no) if**

- Multiple static figures are shown in the paper but no linking is described.
- The system only supports switching between views (tabs) without coordination.

6.3 Libraries: Full vs Partial vs N/A

- **Full**

- The library offers documented, high level mechanisms for coordinated multiple views with radial charts, such as:
 - Subplots or small multiples with shared selections or shared axes.
 - Event callbacks that propagate selection from one radial subplot to others via documented patterns.

- **Partial**

- Multiple views are supported (e.g., facets, small multiples, separate plots), but linking requires non trivial custom code or manual wiring beyond typical high level usage.
- Example: faceted radial charts in Altair/Vega Lite with linking only via custom Vega specifications rather than the default selection grammar.

- **N/A**

- Only single radial views are supported; no documented way to create coordinated multiple radial views (beyond separate static charts).
-

7. Interaction with Dimensions

7.1 Definition

Techniques that allow users to manipulate the dimensions or variables of the data in the radial visualization:

- Dimensional reordering (changing the order of axes/variables).
- Dynamic axis manipulation (scaling, rotating, weighting axes).
- Attribute selection (interactive show/hide of variables).

7.2 Tools/systems: yes vs no

- **Code Interaction with Dimensions = Full** (yes) if

- Users can reorder axes or dimensions interactively at runtime.
- Users can change axis scaling or weighting interactively (e.g., changing axis lengths in Star Coordinates).
- Users can interactively select which dimensions to display using dedicated UI controls (beyond static configuration).
- **Code = N/A (no) if**
 - Dimensions are only defined in code or configuration before rendering, with no runtime end user manipulation.
 - Only simple on/off toggling is allowed through non semantic controls (e.g., turning traces on/off in a legend may be borderline; in the tools context we normally reserve “yes” for more explicit dimension manipulation).

7.3 Libraries: Full vs Partial vs N/A

- **Full**
 - The library provides built in, documented controls for radial axes/dimensions in charts based on the pattern, such as:
 - Axis reordering for radar charts in the UI.
 - Interactive sliders that change dimension weightings with automatic redraw.
 - **Partial**
 - Some dimension interaction is available, but limited:
 - Users can toggle specific series/dimensions via legend visibility, but no continuous reordering/weighting.
 - Interaction is possible but requires substantial custom code (e.g., writing event handlers that reconstruct traces for new dimension orders).
 - **N/A**
 - No documented method for end users to change dimensions at runtime; axes/dimensions are fixed in code.
-

8. Domain Specific Methods

8.1 Definition

Interaction techniques tailored to specific application domains (e.g., genomics, network analysis, temporal event analysis), not just generic zoom/pan/hover.

8.2 Tools/systems: yes vs no

- **Code Domain Specific Methods = Full (yes) if**
 - The system implements interactions explicitly tied to domain semantics, such as:
 - Genomic region selection and annotation in OmicCircos or AuroraGenome.
 - Temporal window selection and playback in SpiraClock or SpiraGraph.
 - Hierarchy aware editing, filtering, or attribute queries in specialized tree/network tools (e.g., PhenoTree, InterRing, Radial Traffic Analyzer).
- **Code = N/A (no) if**
 - Only generic interactions (zoom, pan, hover, generic filtering) are provided without domain specific semantics.

8.3 Libraries: Full vs Partial vs N/A

- **Full**
 - The library ships with dedicated radial modules for specific domains (genomics, network analysis, timelines), where interactions are explained in domain terms and provided as built in configuration.
 - **Partial**
 - Some domain specific behavior is shown in examples or possible via moderate custom code (e.g., using callbacks to implement domain specific selection), but there is no dedicated built in module.
 - **N/A**
 - Library only offers generic charting primitives; any domain specific interaction would require heavy custom development and is not described in documentation.
-

9. Differences between tool and library protocols

You can present this as one unified manual with two short “application sections”:

9.1 Tools/systems protocol (Table 1)

- **Pattern:** assigned once per tool using Section 3; fixed for all interactions.
- **Interaction categories:** for each of the four categories, coder records **Full** or **N/A** based on Sections 5—8 (yes/no rules).
- **Evidence:** tool's research paper, demo videos, and documentation. If a capability is only hinted at but not described, coders should default to **N/A** and note uncertainty.

9.2 Libraries protocol (Table 2)

- **Patterns:** for each library, identify which radial patterns it supports in practice (from documentation: pie/radar Star; sunburst Concentric; radial network Connected Ring, *etc.*).
 - **Interaction categories:** for each supported (pattern, interaction) combination, coder assigns **Full**, **Partial**, or **N/A** using Sections 5—8 (Full/Partial/N/A rules).
 - **Evidence:** official documentation, API references, gallery examples, and tutorials. When the behavior is only possible via low level hacks (*e.g.*, raw D3 in a wrapper), coders lean toward **Partial** or **N/A** depending on the workload and abstraction level.
-

10. Procedural rules for coders

1. Independence

- Each coder applies this manual independently to all tools and library triples, without seeing the other coder's choices.

2. Use of examples

- The example tools and libraries listed in the manual (Hyperbolic Browser, StarCoordinates, Sunburst, *etc.*) serve as anchor cases for each pattern and interaction category; coders compare new cases to these anchors.

3. Handling ambiguity

- If documentation is ambiguous, coders choose the most conservative code (*e.g.*, **Partial** or **N/A** rather than **Full**) and record a brief note.
- During disagreement resolution, coders revisit the original sources and notes and adjust codes if they misinterpreted the taxonomy.

4. Updates to the manual

- During pilot coding, the team can refine wording or add examples, but once the formal IRR phase begins, the manual is fixed to ensure that κ reflects a stable protocol.